
Empirical Application

Brazilian climate data in R

FEA/USP · 2026

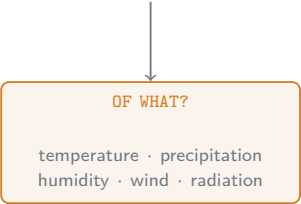
Nara Morais (FEA/USP) · naraliviamorais@usp.br

What we are doing today

The goal here is to **gather, analyze** and **visualize climate data** from Brazil.

What we are doing today

The goal here is to **gather, analyze** and **visualize climate data** from Brazil.

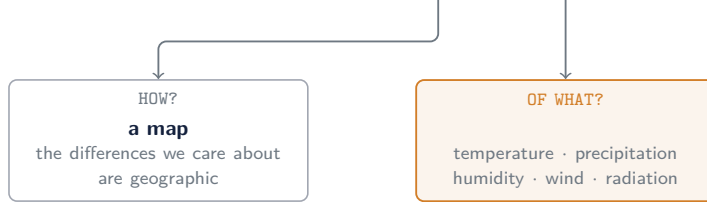


OF WHAT?

temperature · precipitation
humidity · wind · radiation

What we are doing today

The goal here is to **gather, analyze** and **visualize climate data** from Brazil.



What we are doing today

The goal here is to **gather, analyze** and **visualize climate data** from Brazil.



But first, we have to decide four things

- ① **Which product?** A station network, an interpolated grid, a satellite retrieval, a reanalysis. These are different objects, not different accuracies of the same object.
- ② **How to aggregate space?** Grid cells into municipalities — weighted by what? Area, population, cropland, nothing?
- ③ **Which functional form?** Mean, a count above a threshold, degree-days, an anomaly. And: does the averaging happen before or after the transformation?
- ④ **Which boundaries?** Municipalities are created and merged. The year of the map has to match the year of the aggregation.

Source: Dell, Jones & Olken (2014), *JEL*, Sections 2.3–2.4

Part I

Where does Brazilian climate data come from?

The sources

Source	What we observe	Example	From source to municipality
Weather stations	Measurement points	INMET/BDMEP	Assign a station to the municipality, or aggregate the stations
Grids	Spatial cells	Xavier et al. (BR-DWGD)	Intersect the grid with the border and aggregate
Satellites	Surface observation pixels	MODIS/Landsat	Aggregate the pixels inside the border
Reanalyses	Model-based estimates on a grid	ERA5	As for grids — but the values are modelled, not measured

In orange: the source this class uses.

Same last step, whichever door you come in by

Weather stations

INMET / BDMEP

An instrument at a point, reading the air around it.
The catch is which station stands for a municipality
and most municipalities have none.

Grids WHAT WE USE

Xavier et al. (2022) — BR-DWGD

Those same stations interpolated onto a $0.1^\circ \times 0.1^\circ$
lattice, daily. Now every municipality has a value,
including the ones with no thermometer and the
catch is exactly that: the value is estimated.

Satellites

MODIS, Landsat; INPE for Brazil

Pixels, at fine spatial detail. The catch is the variable:
MODIS gives **land surface** temperature . Asphalt at
midday is not what a person or a soybean experiences.

Reanalyses

ERA5 — ECMWF / Copernicus

Observations run through a weather model: hourly,
everywhere, decades back. The catch is that the
values were *computed*.

stations · cells · pixels → municipality → aggregate

Source: INMET/BDMEP; Xavier et al. (2022); MODIS LST (NASA); INPE; ERA5 — ECMWF/Copernicus

Getting it yourself

INSTITUTO NACIONAL DE METEOROLOGIA

Caro(a) usuário(a), seja bem-vindo(a) ao Banco de Dados Meteorológicos do INMET.

ATENÇÃO! Para Download imediato de pacote de dados anuais de TODAS as estações automáticas separadas por ano, [clique aqui](#).

ATENÇÃO! Para coleta de dados horários de curta duração (até 6 meses) de uma estação, é aconselhado o uso da Tabela de Dados de Estação através do sistema [TEMPO](#).

Para escolher uma estação em particular, visualize a distribuição espacial das estações no [Mapa de Estações](#).

Siga as instruções para receber os dados meteorológicos da estação escolhida em formato CSV:

(1) Clique em "Prosseguir" nesta tela, abaixo, e na próxima tela forneça seu endereço de e-mail.

(2) Na tela seguinte, favor preencher todos os campos com atenção: tipo de estação, datas de início e fim, variáveis selecionadas e local. Envie sua solicitação.

Caso seja selecionada uma data inicial anterior à data de início de operação (fundação) de alguma estação, para

BDMEP — `bdmep.inmet.gov.br`

Choose stations, dates, variables. Get one CSV.

Instituto Nacional de Meteorologia
MINISTÉRIO DA AGRICULTURA E PECUÁRIA

Tempo • Clima • Dados Meteorológicos • Satélites • Risco de Incêndio • Previsão Numérica • Sudegra • CTEM • Publicações • Sobre • Informações •

DADOS HISTÓRICOS ANUAIS

ANO 2000 (AUTOMÁTICA)

ANO 2001 (AUTOMÁTICA)

ANO 2002 (AUTOMÁTICA)

ANO 2003 (AUTOMÁTICA)

ANO 2004 (AUTOMÁTICA)

ANO 2005 (AUTOMÁTICA)

Dados Históricos — `portal.inmet.gov.br`

One ZIP per year, 2000 on; one CSV per station.

Source: `bdmep.inmet.gov.br`; `portal.inmet.gov.br/dadoshistoricos`

Good news: there are some known shortcuts

A **shortcut** is somebody having done the previous slide once, properly, and shipped the result as a function call.

- **What it removes.** The download, the encoding, the parsing, the cleaning, the binding of one file per station per year. Some go further and hand you the finished object directly: one number per municipality per day, already aggregated.
- **What it does not remove.** The decisions. Which product, which interpolation, which boundary year, which weighting — all still made, only now made by someone else and no longer visible anywhere in your code.
- **So the question to ask** of any of them is always the same two-parter: *how far along does this hand me the data, and what did it decide on the way there?*

Some suggestions

R

`brclimr`

BR-DWGD and TerraClimate, already averaged to municipalities. **What we use today.**

`rmet`

The raw INMET CSVs, downloaded and cleaned: names standardised across years, timestamps in local time.

`BrazilMet`

Automatic stations — daily, hourly, normals — built for estimating evapotranspiration.

`climateBR`

Municipality centroids matched to their nearest INMET stations.

Python

`inmetpy`

The INMET API: the station list, the nearest stations to a coordinate, and their series.

`xarray + pandas`

No wrapper. You open the gridded file yourself and do the aggregation.

Base dos Dados

`br_inmet_bdmeq`

The BDMEP series as one SQL table on BigQuery, queried from R, Python or straight from the browser.

Why `brclimr`?

`brclimr` hands us daily temperature for a Brazilian municipality — one row per municipality per day, already aggregated. It cuts out the download, the parsing and the spatial work. What it does not cut out is the **decisions**: it makes them for us, upstream, before we type anything.

Decision	What <code>brclimr</code> already chose for us
Product	BR-DWGD — $0.1^{\circ} \times 0.1^{\circ}$, daily, 1961 to July 2020. An <i>interpolation</i> of the station network, not an observation.
Boundary	The 2010 municipality borders from <code>geobr</code> .
Aggregation	A <i>zonal statistic</i> : “descriptive statistics calculated using a set of cells that spatially intersects a given spatial boundary.” Which one — mean, max, min, sd — is left to us.
Weighting	The methodology page does not say. Nothing about how a cell only half inside the border is counted, or whether people in it matter.

Source: rfsaldanha.github.io/brclimr — *Articles > Methodology*

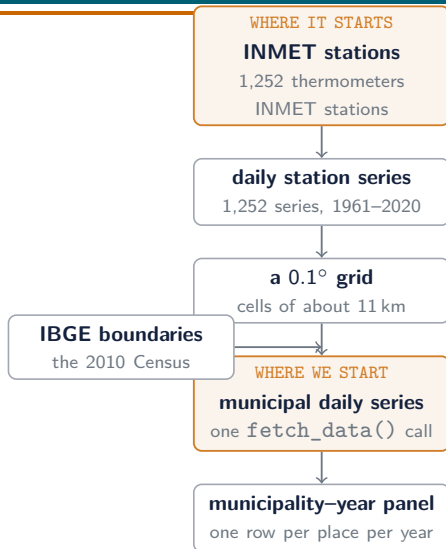
What are we going to do in this exercise?

- ① **Get the data.** One municipality first, then all 5,567 in a single query — and look at what the product actually is: BR-DWGD, an interpolation of the INMET network, not an observation.
- ② **Build the measures.** Annual mean, maximum, and days above a threshold, all from the same daily series — then ask what the threshold should be. This is the one choice the package leaves entirely to us.
- ③ **Put it on a map.** Boundaries from geobr, joined on codes and never on names, at the IBGE 2010 vintage the package already assumed for us — and drawn twice, once against a national mean and once against each state's own.
- ④ **Ask what the mean throws away.** Two cities that share a count and nothing else, and one city fifty-eight years apart. The same daily series, read as a distribution rather than as an average.

Part II

The script

The path until municipal level data



1 · readings → day

Three readings a day at a conventional station, twenty-four at an automatic one, become one row. How many gaps are too many?

2 · interpolate

IDW or ADW — chosen by hiding one station and predicting it from the rest. Every reading corrected to the cell's elevation.

3 · zonal statistics

Each cell weighted by the share of it inside the border, by the people living in it, or by nothing at all.

4 · daily → annual

Mean, days above 30 °C, degree-days. After interpolating, never before: a count is not an average.

Source: Steps 1–3: Xavier, Scanlon, King & Alves (2022), *Int. J. Climatology* 42(16): 8390–8404. We join the chain at step 4.

1. Preambulo - Load the packages

- First step, download and load some packages. If you are new to this, here are some useful guides:
 - R for Data Science by Hadley Wickham and Garrett Golemund
 - Tidyverse by Hadley Wickham
 - Geobr by IPEA
-

```
if (!requireNamespace("pacman", quietly = TRUE)) install.packages("pacman")
pacman::p_load(
  tidyverse,      # dplyr, ggplot2, tidyr, lubridate... the verbs we use all day
  brclimr,        # Brazilian municipal climate data, already aggregated
  arrow,          # reads parquet, and lets dplyr verbs run inside the file
  geobr,          # official IBGE boundaries -- the shapefiles
  sf,             # how R holds a map: a data frame with a geometry column
  glue            # string interpolation, for readable plot labels
)
```

We start getting data from one municipality

```
sp <- brclimr::fetch_data(  
  code_muni = 3550308,      # São Paulo  
  product   = "brdwgd",     # BR-DWGD  
  indicator  = "tmax",  
  statistics = "mean",      # the ZONAL statistic, not a time average  
  date_start = as.Date("2019-01-01"),  
  date_end   = as.Date("2019-12-31"))
```

sp — 365×2

date	value
<date>	<dbl>
2019-01-01	31.17555
2019-01-02	32.16683
2019-01-03	33.47587
2019-01-04	30.19673
... 361 more	

What that call really is

```
con <- DBI::dbConnect(duckdb::duckdb())
DBI::dbExecute(con, "INSTALL httpfs; LOAD httpfs;")

DBI::dbGetQuery(con, "
  SELECT date, value
  FROM 'https://brdwgd.nyc3.cdn.digitaloceanspaces.com/parquet%2Ftmax.parquet'
  WHERE code_muni = 3550308 AND name = 'Tmax_mean'
  AND date BETWEEN '2019-01-01' AND '2019-12-31'")
```

- Same 365 rows, same values — the script checks it with `all.equal()`. The file is **1.9 GB** and never lands on your disk.
- Parquet is columnar and carries statistics in its footer. DuckDB reads that footer over HTTP, works out which blocks *could* match the WHERE, and requests only those byte ranges.
- This is not a trick of any package. It works against any parquet at any public URL.

The whole country, in one query

```
f <- tempfile(fileext = ".parquet")
options(timeout = 3600)
download.file(brclimr::brdwgd_data$tmax$link, f, mode = "wb")

arrow::open_dataset(f) |>
  filter(name == "Tmax_mean",
         (date >= as.Date("1961-01-01") & date <= as.Date("1961-12-31")) |
         (date >= as.Date("2019-01-01") & date <= as.Date("2019-12-31"))) |>
  mutate(year = year(date)) |>
  select(code_muni, year, date, value) |>
  collect() |>
  arrange(code_muni, date) |>
  mutate(value = round(value, 2)) |>
  write_csv("data/derived/tmax_daily_1961_2019.csv.gz")
```

- The same file brclimr reads, fetched once — **1.9 GB**. Everything after that line happens on your own machine.
- `open_dataset()` does not read it. It reads the footer, works out which row groups *could* match, and touches only those: **2 seconds** out of 484,596,216 rows.
- Out comes **4,063,910** rows — 5,567 municipalities × 365 days × 2 years — and a **20 MB** file you can hand to a room full of people.

Source: `climate_data_in_R.R` § 2.2

```
muni <- geobr::read_municipality(code_muni = "all", year = 2010,  
                                simplified = TRUE)  
uf    <- geobr::read_state(year = 2010, simplified = TRUE)  
  
sf::st_crs(muni)      # EPSG:4674 -- SIRGAS 2000, the official datum
```

- An sf object is a **data frame with a geometry column**. Everything you know about dplyr still works — filter, mutate, joins — and the geometry comes along for the ride.
- `simplified = TRUE` returns a generalised outline. The full-resolution coastline is ~100 MB and renders no differently on a projector. Switch it off only for point-in-polygon work.
- `read_state()`, `read_municipality()`, `read_census_tract()`, `read_schools()`, `read_biomes()` ... `list_geobr()` shows all of them.

Source: `climate_data_in_R.R` § 3; Milz (2024), *Dados espaciais com geobr*

So what *is* an `sf` object?

`muni`

```
# Simple feature collection with 5565 features and 8 fields
# Geometry type: GEOMETRY      <- 5,497 POLYGON and 68 MULTIPOLYGON
# Bounding box: xmin -73.99 ymin -33.75 xmax -28.84 ymax 5.27
# Geodetic CRS: SIRGAS 2000
```

code_muni	name_muni	abbrev_state	...	geometry
<dbl>	<chr>	<chr>		<GEOMETRY [°]>
2211001	Teresina	PI	...	POLYGON ((...
3550308	São Paulo	SP	...	POLYGON ((-46.58 -23.37, ...
4106902	Curitiba	PR	...	POLYGON ((...

- **Eight ordinary columns and one strange one.** `geometry` is a *list* column: each cell holds an entire outline. São Paulo's is 541 coordinate pairs — and that is the `simplified = TRUE` version.
- It is not free. `muni` is **21.6 MB**; `st_drop_geometry(muni)` is **674 KB**. Drop it while you are computing, join it back on when you are ready to draw.
- **The CRS is not decoration.** EPSG:4674 is in *degrees*. Any area or distance you compute without `st_transform()` first is wrong — quietly, and by more the further south you go.

Source: `climate_data_in.R` § 3

Join on codes, never on names

The 2020 guide matches municipalities by *name*, lowercased and stripped of accents. That works for one state and 645 rows. Nationally:

```
muni |> st_drop_geometry() |> count(name_muni) |> filter(n > 1)
# 234 names are not unique, covering 508 of 5,565 municipalities
# five "Bom Jesus", five "São Domingos", in five different states
```

```
clim <- unique(tmax_daily$code_muni)
sum(!muni$code_muni %in% clim)      # 0    every polygon has a climate key
sum(!clim %in% muni$code_muni)      # 2    4300001 and 4300002

sum(is.nan(tmax_yearly$mean_temp)) # 18   keys that matched and are still empty
```

- **The keys line up almost perfectly**, and the two that do not are *Lagoa dos Patos* and *Lagoa Mirim* — two lagoons IBGE gave municipality codes to, and geobr has no polygon for either.
- **A left_join never fails**. Those 18 rows are 9 municipalities the grid never covered, Ilhabela among them. The key matched, so nothing warns you; ggplot draws them grey, and grey reads as a missing observation rather than an absent one.
- Count *both* directions before you plot — and then count the empty cells too.

Source: `climate_data_in_R.R` § 3

The same join, drawn

tmax_yearly 11,134 × 7 — 5,567 places × 2 years

code_muni	year	mean_temp	max_temp
2211001	2019	34.19	39.66
3550308	2019	26.96	36.05
4106902	2019	24.62	34.58

muni 5,565 × 5, an sf

code_muni	name_muni	abbrev_state	geometry
2211001	Teresina	PI	POLYGON...
3550308	São Paulo	SP	POLYGON...
4106902	Curitiba	PR	POLYGON...

by = "code_muni"

code_muni	name_muni	geometry	year	mean_temp
3550308	São Paulo	POLYGON...	1961	25.14
3550308	São Paulo	POLYGON...	2019	26.96
3520400	Ilhabela	POLYGON...	2019	NaN

tmax_yearly 11,130 × 11 — one row per polygon per year

- A `left_join` gives you one row per *match*. 5,565 polygons in, 11,130 rows out, because the climate side carries two years. Filter to one year first, or you draw everything twice.
- `geobr` ships its own year — the boundary vintage. The script drops it in the select; leave it in and `dplyr` hands you `year.x` and `year.y` without a word.
- Ilhabela's **NaN** is not a failed join. The key matched; the 0.1° grid never covered the island. **Nine municipalities** are empty this way, and a left join reports none.

Source: `climate_data_in.R` § 3 — this exact join

FEA/USP

A hot day compared to what?

```
ref_br <- daily %>%
  group_by(year) %>%
  summarise(ref_br = mean(value))

ref_state <- daily %>%
  group_by(year, abbrev_state) %>%
  summarise(ref_state = mean(value))

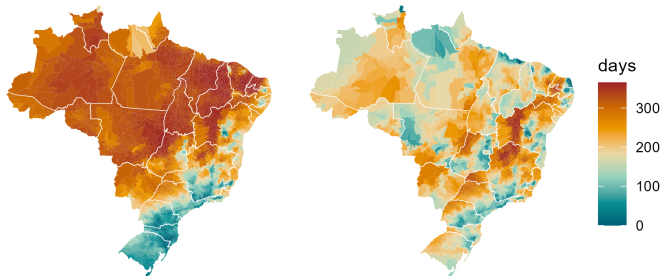
hot <- daily %>%
  left_join(ref_br, by = "year") %>%
  left_join(ref_state,
    by = c("year",
           "abbrev_state")) %>%
  group_by(code_muni, year) %>%
  summarise(
    hot_br = sum(value > ref_br),
    hot_state = sum(value > ref_state))

# map_df = muni + hot(2019), long
ggplot(map_df) +
  geom_sf(aes(fill = n_hot,
    colour = NA) +
  facet_wrap(~ measure) +
  scale_fill_gradientn(colours = SEQ,
    limits = c(0, 365))
```

Hot days in 2019 -- the same data, two references

Above the national mean

Above its own state's mean



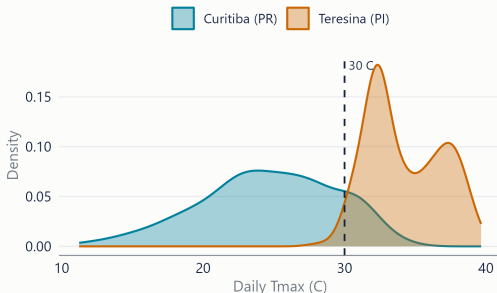
Same data, same year, same colour scale. What changes is **one group_by** — the mean is taken over the country, or over each state. On the left you are looking at Brazil's climate. On the right the state borders become visible, because every municipality is now being compared with the state it sits in.

Source: `climate_data_in.R` § 3.2 and § 4

Two cities, one year

Daily maximum temperature, 2019

The dashed line is the bin. 'hot_days' is the area to its right.



The script does not pick these two. Of the five cities it follows, it takes the *highest* and *lowest* annual `tmean` in 2019 and asks who they are.

	mean $T_{\max}$	sd	≥ 30
Curitiba (PR)	24.6	4.8	57
Teresina (PI)	34.2	2.6	359

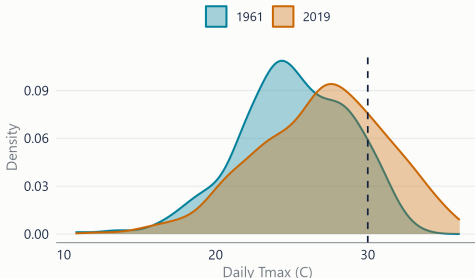
- **hot_days is not a rival to the distribution — it *is* the distribution**, read once, at one place: the area right of the dashed line. Every threshold count in this class is this picture with the curve rubbed out.
- **The hotter city is the narrower one**, and the same count means different things in each: Curitiba's 57 hot days are its entire right tail, Teresina's 359 are its ordinary Tuesday.

Source: `climate_data_in.R` § 5.1

One city, two years

São Paulo (SP): daily $T_{\max}$ in 1961 and 2019

Two years, one place. Look at the mean, then look at the tail.



- **The curve did not slide — it slid *and* widened.** The mean moves 1.8°C , the 90th percentile 2.7 ; days at or above 30°C go from 35 to 92. Report the mean and you have described the smaller half of what happened.
- And the count moved far more than the mean did: $+1.8^{\circ}\text{C}$ on the average, but $\times 2.6$ on the days above 30. **Which number you put in a regression is a claim about which part of the distribution does the damage.**

Same city, same calendar, fifty-eight years apart. CITY_ONE is a knob — put your own city in it and rerun.

	mean $T_{\max}$	sd	p90	≥ 30
1961	25.1	3.7	29.9	35
2019	27.0	4.3	32.5	92
Δ	+1.8	+0.6	+2.7	$\times 2.6$

Source: `climate_data_in_R.R` § 5.2

Closing

What to take away

Concluding remarks

- ① Brazilian climate data is easy to get and easy to misuse. The shortcut that saves you an afternoon has already chosen your product, your boundary year and your spatial weighting — report all three, because `brclimr`'s own methodology page does not.
- ② You almost never need the file. The BR-DWGD parquet is **1.9 GB** and 484,596,216 rows; `arrow` and `DuckDB` read its footer over HTTP, touch only the blocks that could match, and hand back the two years we asked for in **2 seconds**. What ships to this class is 4,063,910 rows in **20 MB**.
- ③ Read the source of small packages. `fetch_data()` is a query against a public URL — once you have seen it written out, you can write it yourself, against any parquet anywhere.
- ④ **“A hot day” is not a fact about the weather. It is a threshold you chose.** The same 2019 data, counted above the national mean and above each state's own mean, are two different maps, and one `group_by` is the entire difference between them.
- ⑤ Warming is not a shift. In São Paulo the mean $T_{\max}$ rose 1.8°C between 1961 and 2019 while days at or above 30°C went from 35 to 92. Report the mean and you have described the smaller half of what happened.
- ⑥ **The join that works is the one to distrust.** Every polygon found a key, and nine municipalities are still empty — Ilhabela among them, because the 0.1° grid never covered the island. `ggplot` draws them in grey and says nothing.

- Xavier, A. C., B. R. Scanlon, C. W. King & A. I. Alves (2022). “New improved Brazilian daily weather gridded data (1961–2020).” *International Journal of Climatology* 42(16): 8390–8404. The grid everything today runs on.
- Schlenker, W. & M. J. Roberts (2009). “Nonlinear temperature effects indicate severe damages to U.S. crop yields under climate change.” *PNAS* 106(37): 15594–15598. Where the threshold count comes from.
- From Class 1: Dell, Jones & Olken (2014), *JEL* 52(3): 740–798. — Jones, Moscona, Olken & von Dessauer (2026), NBER WP 34671.

Packages Saldanha, R. F., *brclimr* — rfsaldanha.github.io/brclimr, *Articles > Methodology*. — Pereira, R. H. M. & C. N. Gonçalves, *geobr* — github.com/ipea/geobr. Also *tidyverse*, *arrow*, *duckdb*, *sf* and *glue*, all loaded through *pacman*.

Learning R Wickham, H. & G. Grolemund, *R for Data Science* — r4ds.had.co.nz; the *tidyverse* itself — tidyverse.org.

Where to go from here

- **The exercise:** `github.com/moraisnara/climate-course` — one script, `exercise/climate_data_in_R.R`, and the one CSV it reads. The deck is in the same repository.
- **Two tutorials in Portuguese, both by Beatriz Milz.**
(2020) “Criando mapas com os pacotes tidyverse e geobr” — `beamilz.com/posts/2020-07-27-criando-mapas-com-os-pacotes-tidyverse-e-geobr`
(2024) “Dados espaciais com geobr”, Amostra R — `beatrizmilz.github.io/2024-09-amostra-geobr`
- **Shown today but not used.** INMET directly — `bdmep.inmet.gov.br` and `portal.inmet.gov.br/dadoshistoricos`. The wrappers around it: `rmet`, `BrazilMet`, `climateBR`, `inmetpy`, and `Base dos Dados (br_inmet_bdmep)`. Other products: `TerraClimate`, `MODIS` and `INPE`, `ERA5` (ECMWF / Copernicus).

Questions?